

advX — Sistema inglese di benchmark e decisione

Esplorazione progettuale per passare da registrazioni di parole isolate alla valutazione del parlato naturale

Documento di lavoro

24 luglio 2026

- [Sintesi](#)
- [1. Obiettivo del sistema](#)
- [2. Situazione di partenza](#)
 - [2.1 Dati disponibili](#)
 - [2.2 Componenti advX già disponibili](#)
- [3. Principi progettuali](#)
 - [3.1 Evidenza e decisione sono responsabilità diverse](#)
 - [3.2 Una distribuzione, non una waveform media](#)
 - [3.3 L'incertezza deve essere esplicita](#)
 - [3.4 Tutto ciò che influenza il risultato è versionato](#)
- [4. Preparazione offline del dataset](#)
 - [4.1 Ingestione e validazione](#)
 - [4.2 Normalizzazione](#)
 - [4.3 Estrazione della rappresentazione](#)
 - [4.4 Split per parlante](#)
 - [4.5 Costruzione del benchmark](#)
- [5. Esempi negativi e confusion set](#)
- [6. Dal database di parole alle frasi naturali](#)
 - [6.1 Perché non usare permutazioni libere](#)
 - [6.2 Metadati lessicali](#)
 - [6.3 Parole target e parole di supporto](#)
 - [6.4 Template grammaticali](#)
 - [6.5 Banca di frasi validate](#)
 - [6.6 Possibile uso futuro di un modello linguistico](#)
 - [6.7 Selezione adattiva](#)
- [7. Acquisizione e forced alignment](#)
 - [7.1 Registrazione](#)
 - [7.2 Perché usare forced alignment](#)
 - [7.3 Contesto acustico](#)
- [8. Differenza tra parola isolata e parola in frase](#)
- [9. Decision engine](#)
 - [9.1 Input](#)
 - [9.2 Soglie](#)
 - [9.3 Output](#)
 - [9.4 Risultato della frase](#)
- [10. Persistenza](#)
 - [10.1 SQLite come catalogo e audit log](#)
 - [10.2 Dove conservare feature e prototipi](#)
- [11. Conservazione o eliminazione del dataset originale](#)

- [12. Metriche di valutazione](#)
 - [12.1 Qualità del benchmark](#)
 - [12.2 Qualità della decisione](#)
 - [12.3 Qualità delle frasi](#)
- [13. Rischi principali e mitigazioni](#)
- [14. Piano sperimentale proposto](#)
 - [Esperimento 0 — Inventario](#)
 - [Esperimento 1 — Benchmark isolato](#)
 - [Esperimento 2 — Decisione calibrata](#)
 - [Esperimento 3 — Frasi controllate](#)
 - [Esperimento 4 — Benchmark continuo](#)
 - [Esperimento 5 — Embedding avanzati](#)
- [15. Roadmap di implementazione condivisa](#)
- [16. Decisioni aperte](#)
- [17. Raccomandazione finale](#)

Sintesi

Questo documento esplora una possibile evoluzione di advX: partire da un dataset nel quale più persone leggono singole parole inglesi, trasformare le registrazioni in benchmark persistenti e usare tali benchmark per valutare le stesse parole all'interno di frasi lette in modo naturale.

La direzione proposta conserva l'idea originaria del progetto — confrontare una lettura con il comportamento medio di un insieme di parlanti — ma sostituisce la singola «media audio» con una rappresentazione statistica più robusta:

- più letture corrette per parola;
- caratteristiche acustiche o embedding versionati;
- uno o più prototipi rappresentativi;
- distribuzioni delle distanze;
- due soglie calibrate, con una fascia di incertezza;
- una decisione spiegabile per ogni parola.

Il sistema non genera permutazioni libere delle parole. Costruisce frasi inglesi sensate attraverso template grammaticali e una banca di frasi validate, scegliendo le parole da esercitare secondo copertura e difficoltà. Poiché il testo letto è noto, un forced aligner può localizzare ogni parola nell'audio continuo. Ogni segmento viene quindi confrontato con il relativo benchmark.

La prima versione riguarda un solo locale inglese e un task esplicito. Il design resta però compatibile con una futura estensione a en-US, en-GB, sillabe Pinyin e toni del mandarino.

1. Obiettivo del sistema

L'obiettivo funzionale è presentare una frase inglese naturale, registrare la lettura dell'utente e produrre un risultato per ciascuna parola target:

```
frase proposta
↓
lettura naturale dell'utente
↓
allineamento testo-audio
↓
segmenti delle singole parole
↓
confronto con benchmark
↓
CORRECT / INCORRECT / UNDECIDABLE / RETRY
```

Il sistema dovrebbe rispondere a quattro domande distinte:

1. L'audio ha qualità sufficiente per essere valutato?
2. La parola prevista è stata effettivamente pronunciata nel punto atteso?
3. Quanto la sua realizzazione è compatibile con le letture corrette del benchmark?
4. L'evidenza è abbastanza forte da formulare una decisione?

Il risultato non deve essere interpretato come una valutazione generale dell'accento o della persona. Misura la compatibilità della singola lettura con un benchmark, per uno specifico locale, task e versione del sistema.

2. Situazione di partenza

2.1 Dati disponibili

L'ipotesi di partenza è un dataset organizzato logicamente in questo modo:

```
parola
└─ speaker_id
   └─ una o più registrazioni della parola
```

L'identità del parlante è importante per evitare contaminazioni tra training e test, ma non deve diventare il bersaglio del modello. Il sistema deve apprendere la variabilità lecita della parola, non riconoscere chi sta parlando.

Prima dell'elaborazione sarà necessario produrre un inventario con almeno:

Campo	Significato
word	Parola letta
speaker_id	Identificatore pseudonimo stabile
audio_path	Posizione temporanea del file originale
locale	Per esempio en-US o en-GB
take_id	Identificatore della registrazione
sample_rate	Frequenza del file sorgente
duration_ms	Durata
source	Provenienza e versione del dataset
consent/license	Base che autorizza l'uso

2.2 Componenti advX già disponibili

Il prototipo attuale dispone già di:

- acquisizione PCM mono a 16 kHz;
- trasporto dei frame tramite Redis Streams;
- aggregazione di finestre audio contigue;
- cleaning deterministico;
- persistenza SQLite di benchmark e tentativi;
- pubblicazione di eventi verso l'interfaccia;
- contratti per un futuro pronunciation engine;
- test unitari, black-box e di performance.

Mancano ancora un motore fonetico reale, la segmentazione del parlato, un worker end-to-end e il decision engine descritto in questo documento.

3. Principi progettuali

3.1 Evidenza e decisione sono responsabilità diverse

L'estrattore acustico deve produrre evidenze: feature, embedding, fonemi osservati, confidence e metriche di qualità. Il decision engine applica regole, policy e soglie per produrre un giudizio.

```

RecognitionEvidence + DecisionContext
    ↓
DecisionEngine
    ↓
DecisionResult

```

Questa separazione consente di cambiare modello acustico senza cambiare il contratto applicativo e permette di ricalibrare le soglie senza rielaborare necessariamente l'intera UI.

3.2 Una distribuzione, non una waveform media

Mediare campione per campione registrazioni di persone diverse non produce un buon riferimento. Le letture differiscono per durata, ritmo, intonazione, intensità e articolazione. Anche dopo l'allineamento, una singola media può rappresentare una voce che non esiste.

Il benchmark di una parola dovrebbe contenere:

- un medoid, cioè una lettura reale molto rappresentativa;
- eventualmente più prototipi per diversi cluster;
- statistiche sulle distanze tra esempi corretti;
- variabilità ammessa;
- soglie calibrate su parlanti non usati per costruire i prototipi.

3.3 L'incertezza deve essere esplicita

Una soglia unica forza il sistema a trasformare casi ambigui in errori o successi. Sono invece necessari almeno quattro stati:

Stato	Significato
CORRECT	Lettura compatibile con il benchmark con evidenza sufficiente
INCORRECT	Discrepanza rilevabile con evidenza sufficiente
UNDECIDABLE	Distanza o confidence nella fascia incerta
RETRY	Audio non adatto alla valutazione

RETRY riguarda l'acquisizione o la qualità. UNDECIDABLE riguarda l'incertezza del modello. Nessuno dei due deve essere mostrato come errore di pronuncia.

3.4 Tutto ciò che influenza il risultato è versionato

Una decisione deve poter essere riprodotta. Devono quindi essere registrate le versioni di:

- cleaning;
- segmentazione o forced alignment;
- estrattore delle feature;
- benchmark;
- calibratore;
- policy decisionale;
- generatore della frase.

4. Preparazione offline del dataset

4.1 Ingestione e validazione

Ogni file viene letto una sola volta dalla pipeline di ingestione e sottoposto a controlli deterministici:

- formato decodificabile;
- canale e sample rate noti;
- durata plausibile;
- assenza di clipping eccessivo;
- quantità minima di voce;
- rapporto segnale-rumore;
- corrispondenza tra parola dichiarata e inventario;
- assenza di duplicati, verificata anche tramite hash.

Il record non valido non viene cancellato silenziosamente: riceve uno stato e una motivazione come T00_SHORT, CLIPPED, NO_SPEECH o CORRUPT_FILE.

4.2 Normalizzazione

Le registrazioni valide vengono convertite nel contratto comune:

```
PCM mono, signed int16 little-endian, 16 kHz
```

Il cleaning deve essere lo stesso, o almeno compatibile e versionato, tra costruzione del benchmark e valutazione online. Differenze sistematiche tra le due pipeline falserebbero le distanze.

Non conviene eliminare silenzi e margini in modo aggressivo: gli attacchi e le code consonantiche sono informativi. Si può conservare un piccolo padding contestuale e registrare la regione vocale stimata.

4.3 Estrazione della rappresentazione

Sono possibili due livelli iniziali.

Baseline interpretabile

- log-mel spectrogram o MFCC;
- normalizzazione per registrazione o parlante;
- Dynamic Time Warping per confrontare durate differenti;
- distanza aggregata e normalizzata per la lunghezza.

Questa soluzione è relativamente semplice e utile per verificare tutta la pipeline.

Evoluzione basata su embedding

Un encoder speech produce una sequenza di vettori o un embedding per parola. Può offrire maggiore robustezza, ma richiede:

- scelta e licenza del modello;
- misurazione dell'informazione legata al parlante;
- validazione sugli accenti rilevanti;
- gestione rigorosa della versione;
- ricalibrazione quando il modello cambia.

Il primo esperimento dovrebbe confrontare entrambe le opzioni sullo stesso split, senza assumere in anticipo che il modello più complesso sia migliore.

4.4 Split per parlante

Lo split deve avvenire per `speaker_id`, non per file:

```
training speakers
  → costruzione di prototipi e distribuzioni

validation speakers
  → scelta delle soglie

test speakers
  → valutazione finale non contaminata
```

Tutte le registrazioni di una persona devono appartenere allo stesso split. Altrimenti il sistema potrebbe ottenere risultati artificialmente buoni grazie alle caratteristiche della voce.

4.5 Costruzione del benchmark

Per ogni chiave:

```
(word, language, locale, task, benchmark_version)
```

la pipeline:

1. raccoglie le letture valide del training set;
2. estrae le rappresentazioni;
3. calcola le distanze tra letture;
4. identifica outlier da revisionare;
5. seleziona un medoid;
6. se necessario costruisce più cluster;
7. calcola statistiche robuste, come mediana e percentili;
8. valuta esempi positivi e negativi del validation set;
9. sceglie soglie di accettazione e rifiuto;
10. congela un artefatto versionato.

Il numero di cluster deve rimanere basso e giustificato dai dati. Cluster eccessivi rischiano di memorizzare i parlanti invece della pronuncia.

5. Esempi negativi e confusion set

Le sole letture corrette consentono un sistema one-class: misurano quanto il tentativo assomigli ai positivi, ma non definiscono bene il confine con gli errori.

Per calibrare INCORRECT sono utili:

- parole foneticamente vicine;
- errori controllati;
- omissioni della consonante finale;
- sostituzioni vocaliche;
- registrazioni della parola sbagliata;
- audio contenente solo rumore o silenzio.

Per l'inglese conviene costruire confusion set specifici. Esempi:

```
ship / sheep  
bit / beat  
cap / cab  
rice / rise  
thin / sin  
light / right
```

La scelta deve dipendere dalle parole realmente presenti e dagli obiettivi educativi. Le metriche vanno riportate anche per confusion set, non soltanto come accuratezza complessiva.

6. Dal database di parole alle frasi naturali

6.1 Perché non usare permutazioni libere

Permutare un elenco produce quasi sempre sequenze scorrette o innaturali. Anche una frase grammaticalmente possibile può essere semanticamente assurda o troppo difficile.

Il problema corretto è una composizione vincolata:

```
massimizzare copertura e valore diagnostico  
soggetto a:  
  grammatica corretta  
  frase sensata  
  lunghezza adatta  
  vocabolario controllato  
  locale coerente
```

6.2 Metadati lessicali

Ogni parola candidata deve avere informazioni minime:


```
{
  "surface": "runs",
  "lemma": "run",
  "part_of_speech": "verb",
  "features": {
    "tense": "present",
    "person": 3,
    "number": "singular"
  },
  "difficulty": 1,
  "locale": "en-US"
}
```

Per nomi serviranno almeno numero e proprietà countable/uncountable; per verbi, forma e valenza di base; per aggettivi e avverbi, gli slot nei quali possono comparire.

Questi metadati possono essere inizialmente curati a mano per un vocabolario ridotto. Un arricchimento automatico dovrà comunque prevedere validazione.

6.3 Parole target e parole di supporto

Non è necessario che tutte le parole della frase abbiano un benchmark. Il generatore distingue:

- **target words**, che saranno valutate;
- **support words**, necessarie per rendere naturale la frase.

Esempio:

```
The small dog runs through the park.
    └── target words ─┘
The / through / the = support words
```

Le support words provengono da una lista controllata. Non contribuiscono al punteggio finché non dispongono di un benchmark adeguato.

6.4 Template grammaticali

Per l'MVP si possono utilizzare template come:

```
The {noun} is {adjective}.
The {noun} {verb} near the {noun}.
I can see a {noun}.
Please {verb} the {noun}.
The {adjective} {noun} {verb}.
```

Ogni slot dichiara vincoli morfologici e semantici. Il sistema applica accordo soggetto-verbo, articoli e forme flesse. Ogni template ha un identificatore e una versione.

6.5 Banca di frasi validate

I template possono risultare ripetitivi. Una banca di frasi revisionate offre maggiore naturalezza:

```
sentence_id
text
locale
difficulty
word_occurrences
review_status
source
```

Il selettore cerca frasi che contengono le parole da esercitare. Se nessuna frase raggiunge la copertura minima, ricorre ai template.

6.6 Possibile uso futuro di un modello linguistico

Un modello generativo potrebbe proporre varianti più ricche, ma l'output non dovrebbe essere accettato direttamente. Un validatore deve controllare:

- presenza esatta delle target words;
- grammatica e lunghezza;
- livello del vocabolario;
- assenza di contenuti inappropriati;
- coerenza con il locale;
- riproducibilità mediante seed e versione;
- approvazione umana prima di entrare nella banca stabile.

6.7 Selezione adattiva

Il compositore non sceglie parole uniformemente. Una possibile priorità è:

```
priorità =
  parole mai valutate
+ parole precedentemente errate
+ parole con risultato indecidibile
+ confusion set utili
- parole già esercitate frequentemente
```

Una frase dovrebbe contenere inizialmente da tre a cinque target words. Inserire troppe parole rende difficile capire quale errore abbia causato il risultato e può rendere la frase artificiale.

7. Acquisizione e forced alignment

7.1 Registrazione

La UI presenta la frase e associa alla sessione:

```
sentence_id
sentence_text
ordered tokens
target occurrences
locale
generator version
```

L'audio arriva attraverso la pipeline Redis già definita. Il worker aggrega i frame fino al completamento della frase, non della singola parola.

7.2 Perché usare forced alignment

Conoscendo il testo, non serve una trascrizione libera. Un forced aligner cerca la collocazione temporale della sequenza prevista e produce intervalli:

Token	Inizio	Fine	Confidence
the	0.10 s	0.23 s	0.89
small	0.24 s	0.57 s	0.94
dog	0.58 s	0.83 s	0.96
runs	0.85 s	1.13 s	0.86

Una confidence troppo bassa non deve produrre automaticamente INCORRECT. Potrebbe indicare parola saltata, testo diverso, rumore o fallimento dell'allineatore.

7.3 Contesto acustico

Nel parlato continuo, i margini della parola contengono coarticolazione. La finestra estratta può includere un padding ridotto, per esempio 50–100 ms, ma:

- il padding deve essere registrato;
- non deve sovrapporre eccessivamente parole vicine;
- benchmark e tentativo devono usare convenzioni compatibili;
- il decision engine deve conoscere la confidence dell'allineamento.

8. Differenza tra parola isolata e parola in frase

La stessa parola cambia quando viene pronunciata nel flusso naturale:

- durata inferiore;
- accento lessicale meno marcato;
- riduzioni vocaliche;
- transizioni influenzate dalle parole vicine;
- assimilazione e linking;
- intonazione dipendente dalla posizione nella frase.

Per questo il benchmark isolato è un punto di partenza, non un oracolo definitivo.

È consigliata un'evoluzione in due fasi:

Fase A — Cold start

Il tentativo estratto dalla frase viene confrontato con il benchmark isolato. Le soglie sono conservative e la fascia UNDECIDABLE è ampia.

Fase B — Benchmark del parlato continuo

Letture validate di parole nelle frasi formano una distribuzione separata, eventualmente condizionata da:

- posizione iniziale, interna o finale;
- fonema precedente e successivo;
- parola accentata o non accentata;
- velocità del parlato.

Non si devono aggiungere automaticamente al benchmark tutti i tentativi classificati CORRECT: un errore sistematico potrebbe auto-rinforzarsi. La promozione richiede criteri più severi e campionamento per revisione.

9. Decision engine

9.1 Input

Il contesto della decisione può essere rappresentato così:

```
{
  "language": "en",
  "locale": "en-US",
  "task": "word_pronunciation",
  "target": "runs",
  "sentence_id": "sentence-42",
  "occurrence": 3,
  "expected_phonemes": ["r", "ʌ", "n", "z"],
  "accepted_variants": [],
  "benchmark_version": "runs-en-US-v1",
  "policy_version": "educational-v1"
}
```

Le evidenze includono:

```
{
  "audio_quality": "ACCEPT",
  "alignment_confidence": 0.91,
  "distance_to_prototypes": [0.18, 0.31, 0.29],
  "nearest_prototype": 0,
  "acoustic_confidence": 0.88,
  "observed_phonemes": ["r", "ʌ", "n", "s"],
  "engine_version": "baseline-mfcc-dtw-v1"
}
```

9.2 Soglie

Per una distanza, valori bassi indicano maggiore compatibilità:

```
distance ≤ accept_threshold
→ candidato CORRECT

accept_threshold < distance < reject_threshold
→ UNDECIDABLE

distance ≥ reject_threshold
→ candidato INCORRECT
```

Prima di applicare le soglie si controllano qualità e allineamento. Un possibile ordine è:

1. audio non valido → RETRY;
2. allineamento insufficiente → RETRY o UNDECIDABLE;
3. evidenza acustica insufficiente → UNDECIDABLE;
4. distanza entro la soglia di accettazione → CORRECT;
5. distanza oltre la soglia di rifiuto → INCORRECT;
6. altrimenti → UNDECIDABLE.

Le soglie non devono essere scelte intuitivamente. Devono essere calibrate sul validation set e congelate con il benchmark.

9.3 Output

```
{
  "status": "INCORRECT",
  "correct": false,
  "score": 0.42,
  "distance": 0.61,
  "confidence": 0.89,
  "reasons": ["FINAL_CONSONANT_MISMATCH"],
  "benchmark_version": "runs-en-US-v1",
  "engine_version": "baseline-mfcc-dtw-v1",
  "policy_version": "educational-v1"
}
```

Il punteggio è utile per la UI, ma non deve essere l'unica informazione persistita. Distanza, confidence, stato e reason codes consentono audit e ricalibrazione.

9.4 Risultato della frase

Il risultato complessivo non dovrebbe essere una semplice media. Una parola facile pronunciata bene non deve nascondere una target word errata.

Una prima policy può riportare:

- numero di target corrette;
- numero di target errate;
- numero di indecidibili;
- numero di retry;
- stato complessivo;
- fluidità separata dalla pronuncia.

Esempio:

```
{  
  "sentence_status": "PARTIALLY_CORRECT",  
  "targets": 4,  
  "correct": 2,  
  "incorrect": 1,  
  "undecidable": 1,  
  "words": []  
}
```

10. Persistenza

10.1 SQLite come catalogo e audit log

SQLite è adatto per metadati, soglie, versioni e risultati strutturati. Una possibile estensione concettuale comprende:

words

```
id, surface, lemma, language, locale, part_of_speech, features_json
```

recordings

```
id, word_id, speaker_id, source_hash, split, quality_status,  
duration_ms, cleaning_version, extractor_version, feature_ref
```

word_benchmarks

```
id, word_id, task, prototype_ref, statistics_json,  
accept_threshold, reject_threshold, sample_count, speaker_count,  
benchmark_version, active
```

sentences

```
id, text, locale, difficulty, source_type, generator_version,  
template_id, review_status
```

sentence_words

```
sentence_id, position, word_id, surface, is_target
```

attempts

```
session_id, sentence_id, word_id, benchmark_id, status, score,  
distance, confidence, reasons_json, engine_version, policy_version
```

10.2 Dove conservare feature e prototipi

Per una baseline piccola, embedding compatti possono essere BLOB SQLite. Sequenze MFCC o molti prototipi possono però far crescere rapidamente il file.

La soluzione consigliata è:

- SQLite per metadati, hash, versioni, soglie e riferimenti;
- artefatti numerici in file immutabili e versionati;
- hash dell'artefatto registrato in SQLite;
- caricamento in memoria o cache Redis durante l'esecuzione.

Il formato esatto va scelto dopo aver misurato dimensione del dataset e pattern di accesso.

11. Conservazione o eliminazione del dataset originale

L'elaborazione produce una rappresentazione derivata, ma questa non sostituisce immediatamente l'audio per tutti gli usi futuri.

Se l'audio viene eliminato non sarà più possibile:

- correggere un errore nel cleaning;
- cambiare feature extractor;
- provare un modello migliore;
- verificare outlier;
- ricostruire i benchmark;
- indagare bias o regressioni.

Prima di cancellare è necessario:

1. verificare che ogni file abbia uno stato;
2. congelare pipeline e versioni;
3. costruire e validare i benchmark;
4. verificare test e metriche su parlanti esclusi;
5. salvare hash, provenienza e licenza;
6. definire retention e procedura di eliminazione;
7. verificare che le feature derivate siano sufficienti;
8. creare, se consentito, un archivio freddo temporaneo.

Anche embedding e caratteristiche vocali possono contenere informazioni sul parlante. Non devono essere considerati anonimi automaticamente. `speaker_id` deve essere pseudonimo, gli accessi limitati e la retention motivata.

12. Metriche di valutazione

12.1 Qualità del benchmark

Per ogni parola:

- numero di parlanti;
- numero di letture valide;
- percentuale di scarti;
- distribuzione delle distanze positive;
- separazione rispetto ai negativi;
- stabilità dei cluster;
- copertura di accenti e condizioni acustiche.

Una parola con pochi parlanti o scarsa separazione deve poter essere marcata NOT_READY.

12.2 Qualità della decisione

Sul test set:

- false acceptance rate;
- false rejection rate;
- percentuale UNDECIDABLE;
- percentuale RETRY;
- accuratezza per parola;
- accuratezza per confusion set;
- metriche per sottogruppo, dove lecito e disponibile;
- calibrazione della confidence.

La percentuale di indecidibili non è semplicemente un fallimento: può essere il costo necessario per evitare giudizi falsamente sicuri. Va però monitorata.

12.3 Qualità delle frasi

- copertura delle parole;
- ripetizione delle stesse strutture;
- lunghezza media;
- percentuale di frasi revisionate;
- tasso di fallimento del forced alignment;
- naturalezza valutata da revisori;
- difficoltà percepita dagli utenti.

13. Rischi principali e mitigazioni

Rischio	Conseguenza	Mitigazione
Benchmark con pochi parlanti	Soglie fragili	Stato NOT_READY, minimo di parlanti
Split casuale per file	Risultati troppo ottimistici	Split esclusivamente per speaker
Waveform media	Riferimento artificiale	Medoid, cluster e distribuzioni
Solo esempi positivi	Confine decisionale debole	Confusion set e negativi controllati
Benchmark isolato usato rigidamente	Falsi errori nel parlato naturale	Soglie conservative e benchmark continui
Errori promossi automaticamente	Contaminazione progressiva	Revisione e criteri severi
Fraasi generate male	Esperienza innaturale	Template, corpus validato, review
Forced alignment incerto	Segmento sbagliato	Confidence e stato retry/undecidable
Eliminazione prematura dell'audio	Benchmark non ricostruibile	Validazione e archivio freddo
Embedding identificativi	Rischio privacy	Pseudonimi, minimizzazione, retention
Score non calibrato	Feedback ingannevole	Soglie validate e reason codes

14. Piano sperimentale proposto

Esperimento 0 — Inventario

Produrre un report senza modificare i dati:

- struttura delle cartelle;
- numero di parole;
- numero di parlanti;
- letture per parola;
- formati e sample rate;

- parole mancanti o sbilanciate;
- disponibilità di locale e licenza.

Gate: scegliere un sottoinsieme con copertura sufficiente.

Esperimento 1 — Benchmark isolato

Su un piccolo insieme di parole:

- applicare cleaning;
- estrarre MFCC/log-mel;
- confrontare con DTW;
- costruire medoid e statistiche;
- eseguire split per parlante;
- misurare separazione tra parole.

Gate: dimostrare che positivi e confusion set sono separabili.

Esperimento 2 — Decisione calibrata

- introdurre accept e reject threshold;
- produrre quattro stati;
- misurare false accept, false reject e undecidable;
- analizzare errori per parola.

Gate: ottenere soglie stabili su parlanti mai visti.

Esperimento 3 — Frasi controllate

- arricchire un vocabolario ridotto;
- scrivere template;
- produrre una banca di frasi;
- registrare letture naturali;
- integrare forced alignment;
- confrontare segmenti con benchmark isolati.

Gate: allineamento affidabile e tasso di retry accettabile.

Esperimento 4 — Benchmark continuo

- raccogliere segmenti validati dalle frasi;
- confrontare distribuzioni isolate e continue;
- costruire prototipi separati;
- ricalibrare le soglie.

Gate: riduzione misurabile dei falsi rifiuti senza aumento eccessivo dei falsi positivi.

Esperimento 5 — Embedding avanzati

Confrontare la baseline con uno o più encoder speech utilizzando gli stessi split e le stesse metriche.

Gate: adottare un nuovo estrattore soltanto se migliora generalizzazione, calibrazione o costo operativo in modo misurabile.

15. Roadmap di implementazione condivisa

Una possibile sequenza di lavoro è:

1. ispezione read-only del dataset;
2. definizione dello schema logico e delle policy di retention;
3. script idempotente di ingestione;
4. feature extractor baseline;
5. builder offline dei benchmark;
6. contratto del decision engine;
7. suite di test deterministica e black-box;
8. metadati lessicali inglesi;
9. compositore di frasi a template;
10. forced alignment;
11. worker end-to-end;
12. calibrazione e validazione;
13. raccolta controllata di parlato continuo;
14. decisione sull'archivio o eliminazione dell'audio.

Ogni fase deve produrre un artefatto verificabile prima di passare alla successiva. In particolare, non è necessario scegliere subito il modello acustico definitivo per stabilizzare contratti, persistenza e decisioni.

16. Decisioni aperte

Prima dell'implementazione devono essere risolti almeno questi punti:

1. Il primo locale sarà en-US o en-GB?
2. Quanti parlanti e quante letture esistono per parola?
3. Il dataset contiene solo letture considerate corrette?
4. Sono presenti metadati su locale o accento?
5. La licenza permette di conservare audio e feature derivate?
6. Le frasi saranno rivolte a bambini, studenti L2 o utenti generici?
7. Qual è la lunghezza massima accettabile di una frase?
8. Il feedback deve indicare fonemi specifici o soltanto la parola?
9. Quale costo è peggiore: accettare un errore o respingere una lettura valida?
10. È necessaria una revisione umana dei casi indecidibili?
11. Quanto spazio occupano le feature rispetto all'audio?
12. Dopo quanto tempo l'audio originale deve essere cancellato?

17. Raccomandazione finale

La direzione è tecnicamente coerente con advX e permette di riutilizzare il database di letture per parola. La prima implementazione non dovrebbe tentare subito una valutazione fonetica completa.

Il percorso più informativo è:

```
inventario del dataset
  → baseline MFCC/log-mel + DTW
  → benchmark a medoid e distribuzioni
  → soglie con fascia indecidibile
  → generazione controllata di frasi
  → forced alignment
  → confronto isolato/continuo
```

Questa sequenza verifica presto l'ipotesi centrale — che le letture per parola contengano informazione sufficiente per distinguere pronunce compatibili e non compatibili — mantenendo sotto controllo privacy, riproducibilità e complessità.

Il primo passo operativo, dopo l'approvazione di questa direzione, sarà quindi un audit read-only del dataset e non una modifica immediata del motore.